SMARTBRIDGE
Let's Bridge the Gap

Smart Internz

SRM UNIVERSITY – AP

Neerukonda, Mangalagiri – 522503, Andhra Pradesh, India

AI-Powered Causal Crime Reconstruction Engine (CCRE)

Advanced Cyber Incident Reconstruction and Graph Analysis System

Project Documentation Report

Submitted by

Name	Roll Number
Mahesh Babu Chittem	AP23110010084
Sai Manvitha Pamulapati	AP23110010001
Pavithra Borra	AP23110010027
Venkata Arjun Cherukuri	AP23110010043
Swetha Pappula	AP23110010016

Section: CSE-H • **III Year / VI Semester**

Under the Supervision of

Mr. Rithvik Konakala Sir

Project Guide

Prepared in partial fulfilment of

SEC 162: Generative AI - II

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

SMARTBRIDGE
Let's Bridge the Gap

Smart Internz

SRM University - AP

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report titled

"AI-Powered Causal Crime Reconstruction Engine (CCRE)"

has been completed as part of the project work by

Name	Roll Number
Mahesh Babu Chittem	AP23110010084
Sai Manvitha Pamulapati	AP23110010001
Pavithra Borra	AP23110010027
Venkata Arjun Cherukuri	AP23110010043
Swetha Pappula	AP23110010016

during Academic Year 2025–2026 under the Department of Computer Science and Engineering, SRM University–AP.

Faculty Signature: _____

Place: Amaravati **Date:** _____

1. Project Description

1. Project Overview

An innovative cybersecurity incident reconstruction tool called the AI-Powered Causal Crime Reconstruction Engine (CCRE) is made to automatically analyse structured security data and recreate whole cyberattack chains. The approach allows analysts to comprehend how an issue happened, how it developed, and what the attacker did by turning disparate security events into a cohesive, evidence-based attack narrative.

Log processing, Retrieval-Augmented Generation (RAG), stringent Large Language Model (LLM) reasoning, graph-based attack modelling, and interactive visualisation are all integrated into CCRE's end-to-end AI-driven forensic pipeline. The system examines security warnings as connected events within a temporal attack sequence and reconstructs their causal linkages instead of handling them as independent signals.

The platform starts by consuming structured security logs that are gathered from network sources, authentication systems, and endpoints. These logs are organised into a rigid chronological timeline and normalised into standardised event objects, which serve as the foundation for any further reasoning. A FAISS-based retrieval layer is then used to enhance this timeline by retrieving pertinent threat intelligence and past attack context from a localised cybersecurity knowledge base.

A strictly limited LLM reasoning engine driven by Ollama (LLaMA3) receives the augmented context and event timeline. The model uses evidence-based causal reasoning to construct organised attack relationships in deterministic JSON format, identify probable attack progression, and infer the connections between events. Before being approved, all created entities and causal connections are verified against the initial timetable to guarantee dependability.

Following validation, the reconstructed linkages are converted into a Directed Acyclic Graph (DAG), which enables analysts to see the attack chain, pinpoint crucial attack routes, and separate malicious activity from background noise. An interactive frontend dashboard and an AI-generated Security Operations Center (SOC) report that provides a concise, analyst-friendly summary of the incident are used to display the final product.

Graph intelligence, threat-context retrieval, deterministic AI reasoning, and stringent validation are all combined in CCRE, which greatly minimises the amount of manual investigative work and offers a scalable basis for digital forensics and next-generation cyber incident reconstruction.

2. Problem Statement

Because modern cyberattacks are more often multi-stage, stealthy, and widely dispersed, it is challenging to use traditional security monitoring techniques to examine them. Large amounts

of security logs are constantly produced in enterprise settings by network devices, cloud infrastructure, endpoints, and authentication systems. Even though these logs have important forensic information, it is challenging to piece together the exact order of an attack because they are frequently noisy, disjointed, and fragmented.

Rather than recreating attack narratives, traditional Security Information and Event Management (SIEM) systems are more concerned with creating warnings. These systems are good at identifying isolated abnormalities, but they frequently don't explain the relationship between several alarms, the course of an attack, or the specific activities that led to the compromise. Because of this, analysts must manually review thousands of warnings, correlate logs from various systems, and rebuild attack chains using laborious and prone to error forensic methods.

Several significant obstacles are introduced by this manual inquiry procedure. In addition to increasing analyst burden and delaying incident response, it significantly increases reliance on human expertise for determining attack progression and underlying cause. Alert weariness further decreases productivity in large-scale settings, leading to the overlooking or incorrect interpretation of crucial connections between events. Additionally, current workflows find it difficult to deliver consistent, fact-based explanations that analysts can utilise for quick reporting and decision-making.

An intelligent cyber incident reconstruction system that can automatically convert unstructured security logs into structured attack narratives, deduce significant connections between events, reconstruct the entire attack chain, and produce trustworthy, analyst-ready incident explanations with little human intervention is required to overcome these constraints.

3. Objectives

The creation of an intelligent cybersecurity incident reconstruction platform that can automatically convert unstructured security data into organised, evidence-based attack narratives is the main goal of the AI-Powered Causal Crime Reconstruction Engine (CCRE). By reconstructing how a cyberattack happened, how it developed, and which actions were causally responsible for the compromise, the method aims to minimise manual forensic labour.

Building a dependable end-to-end processing pipeline that ingests structured security logs, normalises them into standardised event objects, and arranges them into a tight chronological timeline is one of the project's main goals. This timeline guarantees that every further analysis is evidence-based and consistent over time by acting as the foundation for downstream reasoning.

Implementing a Retrieval-Augmented Generation (RAG) layer, which improves incident understanding by obtaining pertinent threat intelligence and past attack patterns from a localised cybersecurity knowledge base, is another important goal. Instead of depending only on isolated event data, this allows the machine to base its reasoning on contextual evidence.

Additionally, the project intends to incorporate a rigorously limited Large Language Model

(LLM) reasoning engine that can generate organised attack relationships in deterministic JSON format, identify anticipated attack progression, and infer causal linkages between events. This enables the system to carry out intelligent cause reconstruction in addition to solitary alert analysis.

Creating graph-based attack structures that visually depict incursion evolution, emphasise crucial attack paths, and assist analysts in separating significant malicious activity from background noise is another goal.

The project's ultimate goal is to provide Security Operations Center (SOC) reports that are ready for analysts and provide a concise, human-readable summary of the reconstructed incident. This will speed up investigations, lessen analyst workloads, and facilitate quicker, evidence-based incident response decisions.

4. Core Technologies Used

Technology	**Purpose**
Python	Core backend programming language used for log processing, orchestration, causal reasoning, validation, and API integration.
FastAPI	High-performance backend framework used for asynchronous API development, request handling, and service orchestration.
Pydantic	Schema validation framework used to enforce strict data models on incoming logs, parsed events, and generated outputs.
React + Vite	Frontend framework used for building a fast, responsive, and component-driven analyst dashboard.
Cytoscape.js	Graph visualization library used to render interactive attack graphs and Directed Acyclic Graphs (DAGs).
Dagre	Directed graph layout engine used to arrange attack graph nodes in a clear chronological left-to-right flow.
Neo4j	Graph database used for storing, traversing, and analyzing structured event relationships and attack chains.
FAISS	High-speed vector similarity engine used for semantic retrieval of historical attack patterns and threat intelligence.
Sentence Transformers	Embedding model framework used to convert incident text into dense vector representations for retrieval.
Ollama (qwen2.5)	Local Large Language Model inference engine used for evidence-based causal reasoning.

5. Key Features

Security Log Ingestion

- Accepts structured JSON security logs from multiple sources such as endpoints, network devices, authentication systems, and cloud services.
- Supports ingestion of heterogeneous security events for centralized incident reconstruction.

Event Extraction and Normalization

- Converts raw security logs into a standardized internal event schema for consistent processing.
- Extracts critical metadata such as timestamp, user identity, source, destination, action type, and severity.

Retrieval-Augmented Threat Context

- Retrieves relevant historical attack patterns and MITRE-aligned threat intelligence using FAISS-based semantic search.
- Enriches incident context before reasoning to improve accuracy and reduce unsupported inference.

Strict LLM-Based Causal Reasoning

- Uses Ollama (LLaMA3) to infer evidence-based causal relationships between timeline events.
- Enforces strict prompt rules and deterministic JSON output to ensure reliable structured reasoning.

Attack Graph Construction

- Constructs a Directed Acyclic Graph (DAG) where nodes represent events and edges represent validated causal links.
- Visually reconstructs attack progression for investigation and critical path analysis.

Critical Path Extraction

- Identifies the most significant sequence of causally connected malicious events.
- Helps analysts focus on the primary attack path and root cause of compromise.

AI-Powered SOC Report Generation

- Generates analyst-ready Security Operations Center (SOC) incident reports in human-readable language.
- Summarizes attack flow, root cause, severity, and recommended response actions.

Interactive Analyst Dashboard

- Provides an interactive dashboard for visualizing attack graphs, timelines, and incident reports.
- Enables node-level inspection, timeline analysis, and graph-based exploration of reconstructed attacks.

6. Target Users

- Security Operations Center (SOC) Analysts responsible for monitoring, investigating, and responding to security incidents.
- Incident Response Teams that require rapid reconstruction of cyber attacks for containment and remediation.
- Threat Hunting Teams that proactively search for suspicious behavior and hidden attack patterns within enterprise environments.
- Digital Forensics Investigators analyzing compromised systems, attack chains, and evidence trails after security breaches.
- Enterprise Cybersecurity Teams managing large-scale security monitoring, threat intelligence, and organizational defense operations.
- Security Researchers and Cyber Defense Engineers studying attacker behavior, intrusion techniques, and advanced cyber incident patterns.

7. Real-World Applications

The CCRE platform is intended for use in real-world cybersecurity settings where prompt and precise analysis of significant security events is required. It is very useful in cyber threat intelligence workflows, digital forensic investigations, enterprise incident response teams, and Security Operations Centers (SOCs).

Phishing attacks, insider threats, privilege escalation incidents, ransomware installations, lateral movement scenarios, and data exfiltration campaigns can all be recreated using the technology. CCRE greatly lessens the workload of analysts and enhances the speed, clarity, and calibre of security investigations by automatically linking isolated events into important attack chains.

Application Scenarios / Use Cases

Scenario 1: Phishing-Based Credential Compromise

A phishing email with a fraudulent login link is sent to a user. The attacker gains unauthorised access by stealing legitimate credentials. CCRE allows analysts to swiftly determine the root cause by reconstructing the entire chain from phishing distribution to credential theft, suspicious login, and account penetration.

Scenario 2: Insider Threat Investigation

An inside employee uploads private information to an unauthorised external location and accesses sensitive files during regular business hours. CCRE reconstructs the insider threat path for forensic investigation, connects the file access sequence, and finds anomalous access patterns.

Scenario 3: Privilege Escalation and Lateral Movement

Before going laterally across systems, an attacker obtains access to a low-privilege account and progressively increases rights. Through graph-based attack reconstruction, CCRE monitors the development from initial access to privilege escalation, internal movement, and system compromise.

Scenario 4: Data Exfiltration Detection

Sensitive company assets are accessed by a compromised account, which also starts sizable outgoing payments. CCRE highlights the exfiltration event as the crucial end point of the attack chain, reconstructs the attack path, and detects the series of suspicious behaviours.

2. Technical Architecture

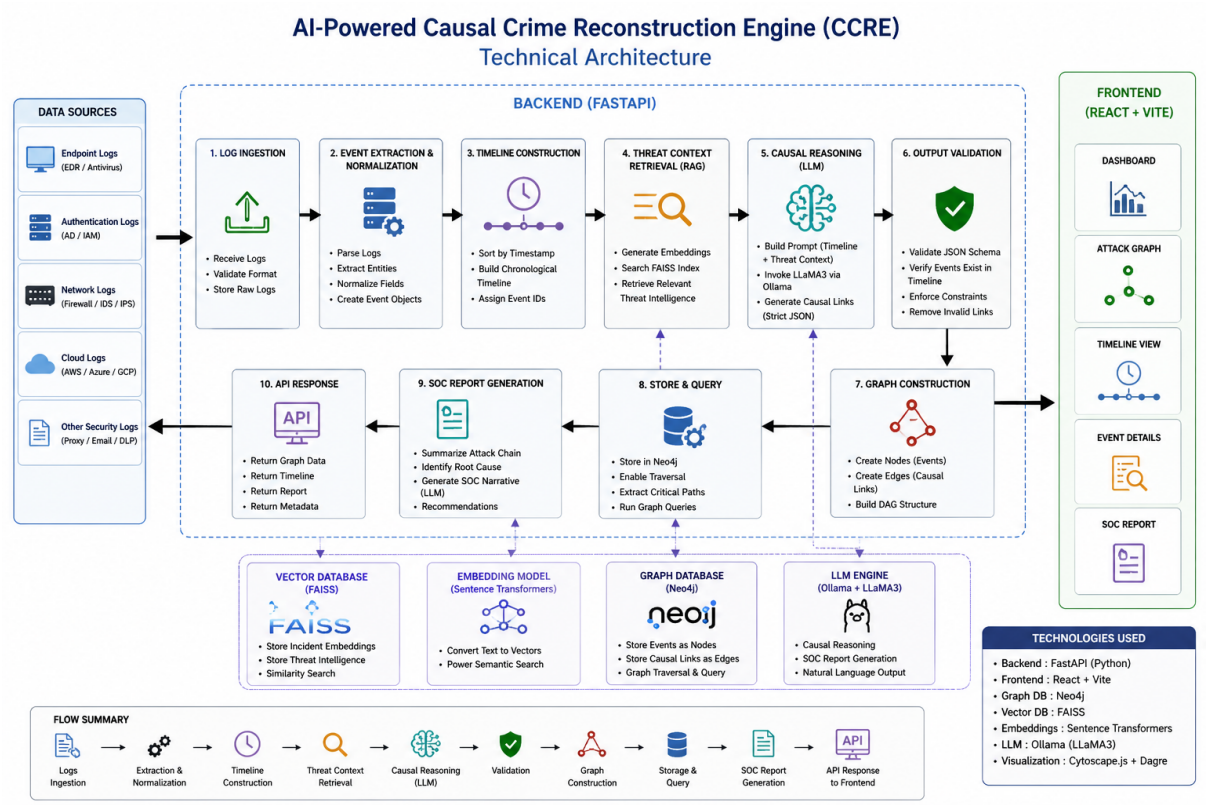


Figure 1: High-Level Technical Architecture of AI-Powered Causal Crime Reconstruction Engine (CCRE)

Frontend Layer

- Built using React and Vite for fast, responsive, and modular user interface development.
- Provides an interactive analyst dashboard for uploading logs, viewing attack graphs, and inspecting incident timelines.
- Uses Cytoscape.js for rendering interactive attack graphs and Dagre for structured graph layout.
- Supports split-pane visualization with graph view, timeline analysis, and SOC report display.

Backend Layer

- Built using FastAPI to manage API requests, orchestration, and end-to-end pipeline execution.

- Handles log ingestion, event normalization, timeline construction, prompt generation, and response delivery.
- Coordinates retrieval, reasoning, validation, graph construction, and report generation.
- Uses Pydantic for schema validation and strict structured data handling.

Graph Database Layer

- Uses Neo4j to store structured event relationships as graph nodes and edges.
- Supports graph traversal, attack-path querying, and critical path extraction.
- Maintains structured persistence for validated attack chains and graph-based investigation.

Vector Retrieval Layer

- Uses FAISS for high-speed semantic retrieval of threat intelligence and historical attack patterns.
- Stores dense vector embeddings generated from cybersecurity knowledge and incident context.
- Retrieves relevant contextual information to enrich downstream reasoning and reduce unsupported inference.

AI / LLM Layer

- Powered by Ollama (LLaMA3) for evidence-based causal reasoning and SOC narrative generation.
- Infers causal links between events using timeline-grounded and context-aware reasoning.
- Produces deterministic JSON outputs and analyst-ready natural language incident reports.

Deployment Environment

- Designed for modular deployment across local, on-premise, and cloud environments.
- Supports isolated deployment of frontend, backend, graph database, vector retrieval, and LLM services.
- Can be containerized for scalable enterprise deployment and secure infrastructure integration.

3. High-Level Architecture Diagram

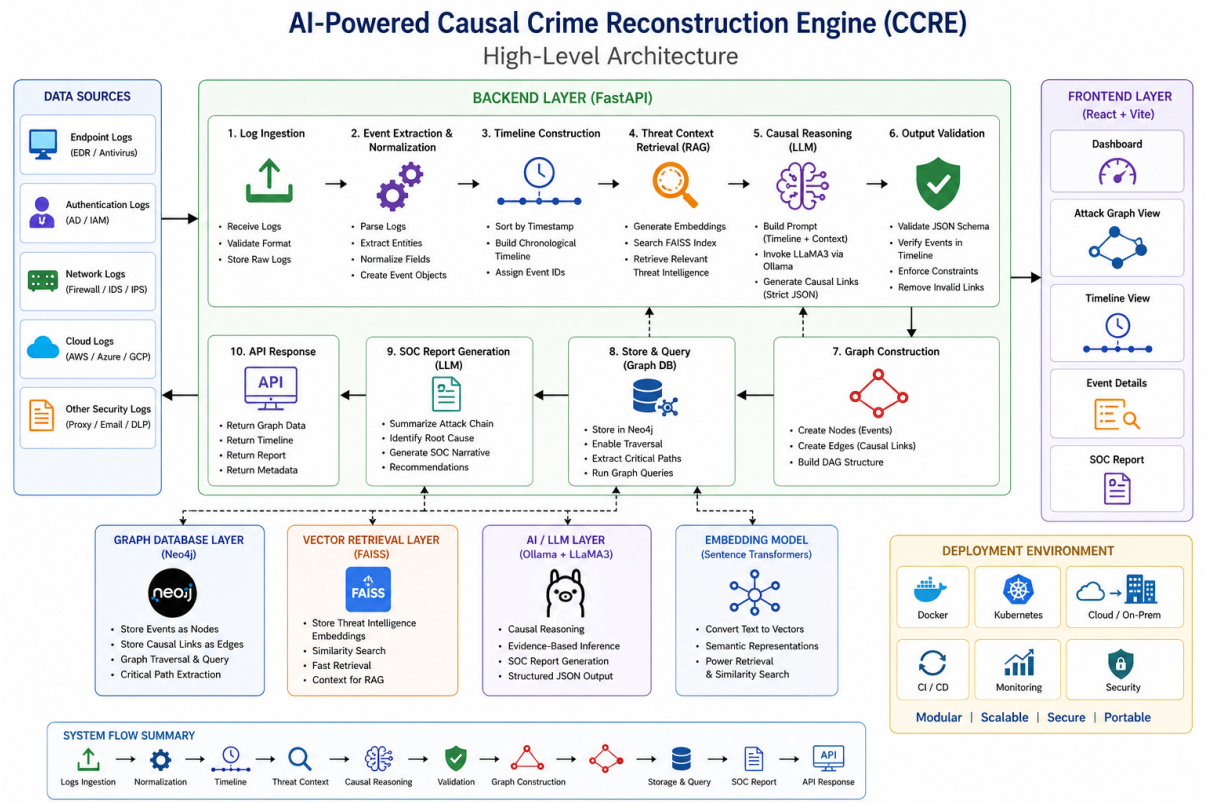


Figure 2: High-Level Architecture of AI-Powered Causal Crime Reconstruction Engine (CCRE)

4. Component Interaction Diagram

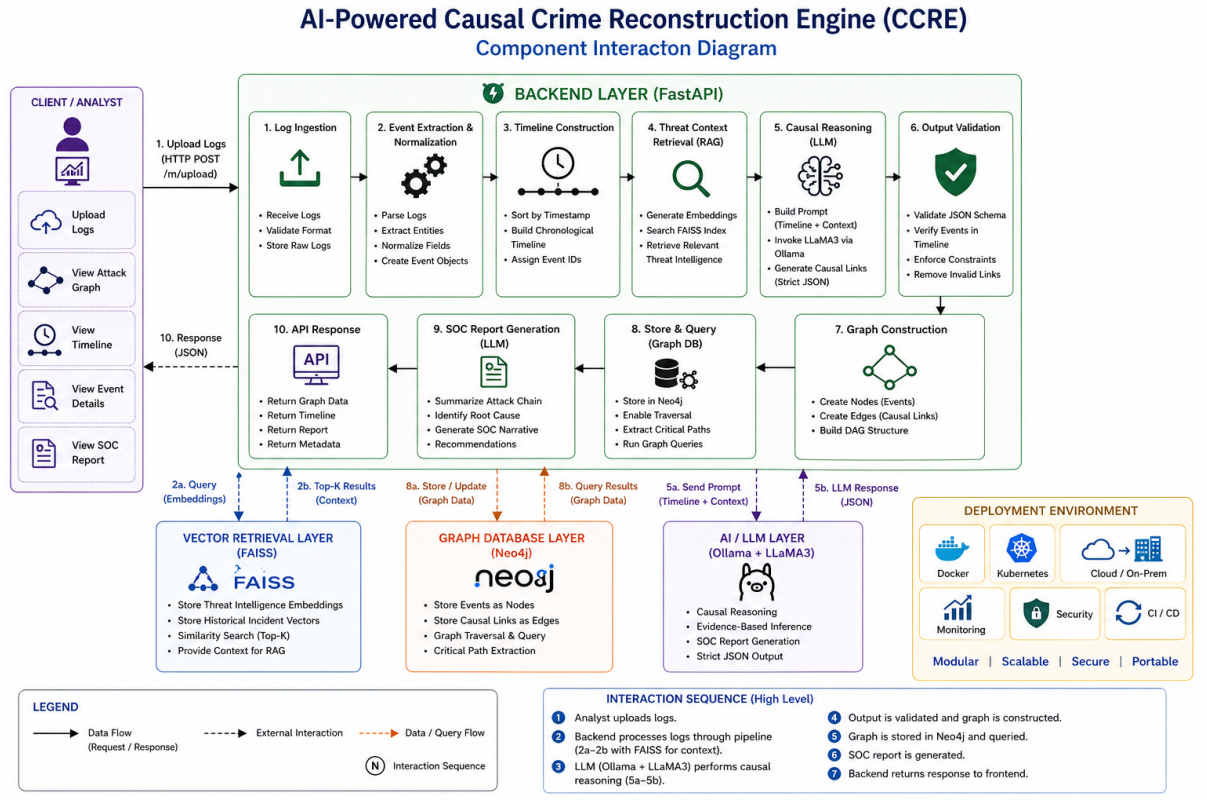


Figure 3: Component Interaction Diagram of AI-Powered Causal Crime Reconstruction Engine (CCRE)

5. Pre-requisites

Software Requirements

- Python 3.9 or higher for backend development, event processing, and AI integration.
- Node.js 18 or higher for frontend development and dependency management.
- Neo4j for graph storage and relationship traversal.
- Visual Studio Code for backend and frontend development.
- Git for version control and collaborative development.
- Modern web browser (Chrome / Edge / Firefox) for dashboard access.

Libraries / Frameworks / Dependencies

```
pip install fastapi uvicorn pydantic neo4j faiss-cpu sentence-transformer
npm install react vite cytoscape dagre axios
```

- FastAPI for backend API development and service orchestration.
- Uvicorn as the ASGI server for running the backend.
- Pydantic for schema validation and structured data enforcement.
- Neo4j Driver for graph database communication.
- FAISS for semantic vector retrieval and threat-context search.
- Sentence Transformers for generating dense semantic embeddings.
- Ollama for local LLM inference and SOC report generation.
- React for building the frontend analyst interface.
- Vite for fast frontend development and rendering.
- Cytoscape.js for interactive attack graph visualization.
- Dagre for structured graph layout rendering.
- Axios for frontend-backend API communication.

Hardware Requirements

- Intel Core i5 / AMD Ryzen 5 or higher processor.
- Minimum 8 GB RAM for smooth local execution.
- Minimum 5 GB free storage for dependencies and assets.
- Optional NVIDIA GPU for faster embedding and LLM inference.
- Stable internet connection for package installation and updates.

6. Prior Knowledge Required

Programming Fundamentals

- Strong understanding of Python fundamentals including functions, modules, and object-oriented programming.

- Basic JavaScript knowledge for frontend interaction and API integration.
- Familiarity with JSON data structures for log parsing and event handling.
- Basic debugging, exception handling, and modular code development.

Framework Knowledge

- Understanding of FastAPI for backend API development.
- Knowledge of REST APIs, request/response flow, and HTTP methods.
- Familiarity with React component-based frontend development.
- Basic understanding of asynchronous programming concepts.

Database Knowledge

- Basic understanding of databases and data modeling.
- Familiarity with graph databases and relationship-based querying.
- Knowledge of Neo4j for graph storage and traversal.
- Basic understanding of structured event and relationship modeling.

AI / ML Fundamentals

- Basic understanding of embeddings and semantic similarity.
- Familiarity with Retrieval-Augmented Generation (RAG).
- Basic knowledge of Large Language Models (LLMs).
- Understanding of prompt engineering and structured AI output.

Cybersecurity Basics

- Understanding of logs, alerts, threats, and attack chains.
- Familiarity with SIEM workflows and incident response.
- Basic knowledge of phishing, privilege escalation, and data exfiltration.
- Awareness of digital forensics and cyber investigation workflows.

7. Project Objectives

Technical Objectives

- Design and develop a scalable AI-powered cyber incident reconstruction platform using FastAPI, Neo4j, FAISS, and Ollama.
- Implement structured security log ingestion and event normalization for consistent downstream analysis.
- Build a strict evidence-based reasoning pipeline for identifying causal relationships between security events.
- Construct graph-based attack chains to represent intrusion progression and support critical path analysis.
- Integrate Retrieval-Augmented Generation (RAG) for contextual threat intelligence retrieval and incident grounding.
- Generate analyst-friendly SOC incident reports using Large Language Models for natural language explanation.

Performance Objectives

- Ensure fast backend response times for log ingestion, event processing, and graph generation.
- Maintain efficient graph traversal and attack path analysis for large volumes of security events.
- Achieve reliable semantic retrieval for historical threat context using vector embeddings.
- Minimize latency in AI-generated SOC reports and incident summaries.
- Provide scalable performance for enterprise-scale security investigations.

Deployment Objectives

- Enable modular deployment of frontend, backend, graph database, retrieval, and AI components.
- Support cloud-based, on-premise, and hybrid deployment models for enterprise environments.
- Ensure secure handling of configuration, credentials, and environment variables.

- Support scalable deployment using containerized infrastructure and service isolation.
- Maintain reliable availability for continuous security analysis workflows.

Learning Outcomes

- Gain practical experience in building AI-powered cybersecurity systems.
- Understand integration of graph databases, vector retrieval, and generative AI in real-world applications.
- Learn how structured reasoning can be applied to cyber incident reconstruction.
- Develop hands-on skills in API design, system orchestration, and security data processing.
- Improve understanding of AI-assisted digital forensics and intelligent incident response systems.

8. Project Workflow

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

- Identify the need for an intelligent cyber incident reconstruction system capable of automatically analyzing security logs and reconstructing attack chains.
- Address the limitations of traditional SIEM workflows that generate disconnected alerts without explaining attack progression.
- Define a solution that combines structured log processing, threat-context retrieval, AI reasoning, and graph-based reconstruction.

Activity 1.2: Functional Requirements

- FR-01: Accept structured security logs from multiple sources for centralized analysis.
- FR-02: Normalize logs into structured event objects for downstream processing.
- FR-03: Construct chronological attack timelines from normalized events.
- FR-04: Retrieve contextual threat intelligence using semantic search.
- FR-05: Infer causal relationships between events using strict AI reasoning.

- FR-06: Generate graph-based attack chains and critical path views.
- FR-07: Produce analyst-ready SOC incident reports.
- FR-08: Expose backend APIs for ingestion, analysis, and report generation.

Activity 1.3: Non-Functional Requirements

- NFR-01 Performance: Ensure low-latency processing for incident reconstruction workflows.
- NFR-02 Reliability: Maintain consistent structured outputs and validated causal reasoning.
- NFR-03 Scalability: Support large-scale security events and enterprise log volumes.
- NFR-04 Security: Protect configurations, credentials, and internal threat intelligence sources.
- NFR-05 Maintainability: Follow modular architecture for frontend, backend, retrieval, and AI services.
- NFR-06 Usability: Provide analyst-friendly outputs with intuitive graph and report interfaces.

Activity 1.4: System Design Decisions & Technology Stack Selection

- Backend Framework: FastAPI for asynchronous backend orchestration and API handling.
- Frontend Framework: React + Vite for responsive analyst dashboard development.
- Graph Storage: Neo4j for event relationship storage and attack-path traversal.
- Retrieval Layer: FAISS for semantic threat-context retrieval.
- AI Reasoning Engine: Ollama (LLaMA3) for evidence-based causal reasoning and SOC reporting.
- Visualization: Cytoscape.js + Dagre for graph rendering and structured layout.

Milestone 2: Environment Setup & Initial Configuration

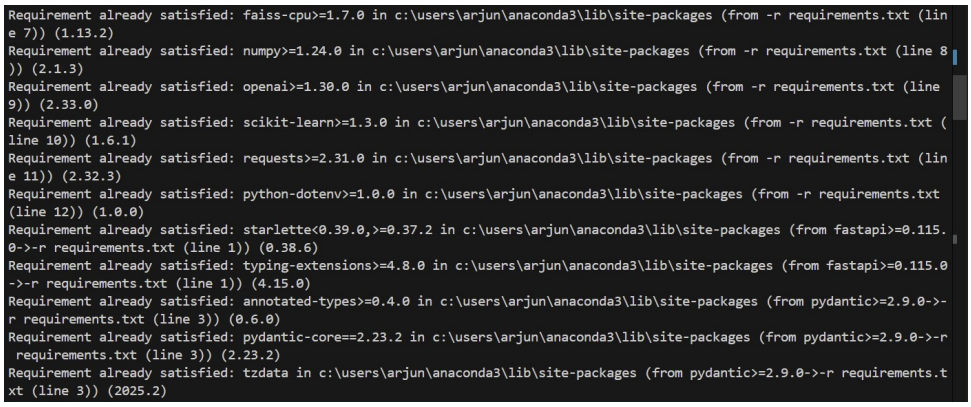
Activity 2.1: Development Environment Setup

- Install Python 3.9+ and Node.js 18+ for backend and frontend development.

- Configure Visual Studio Code with required extensions for Python, JavaScript, and API development.
- Create and organize the project workspace with separate frontend and backend modules.
- Initialize isolated virtual environments and frontend package management.

Activity 2.2: Dependency Installation

- Install backend dependencies including FastAPI, Uvicorn, Pydantic, Neo4j, FAISS, Sentence Transformers, and Python-Dotenv.
- Install frontend dependencies including React, Vite, Cytoscape.js, Dagre, and Axios.
- Configure Ollama locally for LLaMA3-based inference and report generation.



Activity 2.3: Project Structure Creation

- Organize the project into modular directories for frontend, backend, retrieval, graph logic, and utilities.
- Separate routes, services, schemas, components, and configuration files for maintainability.
- Establish reusable project structure for scalable development and deployment.

Activity 2.4: Configuration Setup

- Configure environment variables for Neo4j, FAISS, backend settings, and local model access.
- Store credentials securely using `.env` configuration files.
- Set up application configuration for modular and secure runtime execution.

Milestone 3: Data Infrastructure Setup

Activity 3.1: Create Neo4j Graph Database Project

Step 1: Set Up Neo4j Database

1. Install Neo4j Desktop or configure a Neo4j local/server instance.
2. Create a new Neo4j project for storing security events and causal relationships.
3. Configure database credentials and create a graph workspace.
4. Start the Neo4j database instance and verify active status.

Step 2: Verify Database Status

1. Open Neo4j Browser from the project dashboard.
2. Verify database connection and running state.
3. Ensure the graph service is active and accessible for backend integration.

```
        self.index = faiss.IndexFlatIP(self.dim)
        self.documents = []

        # Threshold for semantic search (cosine similarity 0 to 1)
        self.similarity_threshold = 0.4

        self._initialize_knowledge()

    def _get_model(self):
        if self.model is None:
            try:
                from sentence_transformers import SentenceTransformer
                self.model = SentenceTransformer("all-MiniLM-L6-v2")
            except Exception as e:
                logger.error(f"Failed to load sentence_transformers: {e}")
        return self.model

    def _chunk_text(self, text: str, chunk_size: int = 150, overlap: int = 30) -> list[str]:
        """
        Split a large document into overlapping chunks by word count.
        Using larger chunks to capture more semantic meaning per chunk.
        """
        words = text.split()
        chunks = []
        for i in range(0, len(words), max(1, chunk_size - overlap)):
            chunk = " ".join(words[i:i + chunk_size])
            chunks.append(chunk)
            if i + chunk_size >= len(words):
                break
        return chunks
```

Activity 3.2: Create Graph Structures

1. Define graph schema for storing event nodes and causal relationships.
2. Create labels for security events and relationship types for causal links.
3. Configure graph queries for event traversal, attack-path extraction, and critical path analysis.
4. Verify graph structure creation using Neo4j Browser queries.

Activity 3.3: Store Configuration Keys in `.env`

1. Create a `.env` file in the project root directory.
2. Store Neo4j URI, username, password, backend configuration, and model settings securely.
3. Add FAISS configuration paths and Ollama runtime settings.
4. Verify secure access for backend, retrieval, and AI services through environment variables.

Milestone 4: Core System Development

Activity 4.1: Security Log Ingestion & Event Processing Module

This module is responsible for accepting structured security logs from multiple sources and transforming them into standardized event objects for downstream analysis. It captures security-relevant fields such as timestamps, actors, actions, assets, and severity, ensuring that all incoming logs are normalized into a consistent internal schema. The functionality is implemented within the backend ingestion and preprocessing modules, where logs are validated, parsed, and converted into structured events for timeline construction.

```
    # Normalize severity
    sev = str(output["severity"]).strip().capitalize()
    valid_severities = {"Low", "Medium", "High", "Critical"}
    if sev not in valid_severities:
        sev = "Medium"
    output["severity"] = sev

    # Normalize confidence (handle strings like "0.8" or "80%")
    conf = output["confidence"]
    try:
        if isinstance(conf, str):
            conf = conf.replace("%", "").strip()
            conf = float(conf)
        if isinstance(conf, (int, float)):
            if conf > 1.0:
                conf = conf / 100.0
            output["confidence"] = round(max(0.0, min(1.0, conf)), 2)
        else:
            output["confidence"] = 0.0
    except Exception:
        output["confidence"] = 0.0

    return output
```

Activity 4.2: Threat Context Retrieval & Analysis Engine

This component processes normalized event timelines and retrieves relevant threat intelligence using semantic similarity search. It identifies related historical attack patterns, MITRE-aligned intelligence, and contextual attack signatures to enrich incident understanding before reasoning begins. The logic is implemented in the retrieval layer using FAISS and sentence embeddings, where relevant threat context is fetched, filtered, and prepared for integration into the reasoning pipeline.

```
        # — Normalize Edges —
        normalized_edges = []
        normalized_causal_graph = []  # Legacy compat

        for edge in raw_edges:
            from_node = None
            to_node = None
            reason = "causal link"
            weight = 0.7

            if isinstance(edge, dict):
                from_node = edge.get("from", edge.get("source", "unknown"))
                to_node = edge.get("to", edge.get("target", "unknown"))
                reason = edge.get("reason", edge.get("label", "causal link"))
                raw_weight = edge.get("weight", edge.get("confidence", None))
                # Handle "strength" field (Strong/Moderate/Weak) from new schema
                strength = edge.get("strength", "")
                if raw_weight is not None:
                    try:
                        weight = float(raw_weight)
                        weight = max(0.0, min(1.0, weight))
                    except (ValueError, TypeError):
                        weight = 0.7
                elif strength:
                    weight = 0.85 if strength == "Strong" else 0.65 if strength == "Moderate" else 0.45
                else:
```

Activity 4.3: AI Reasoning & SOC Report Generation System

This module integrates Ollama (LLaMA3) to perform evidence-based causal reasoning over structured event timelines and retrieved threat context. It infers causal relationships, reconstructs likely attack progression, and generates deterministic structured outputs for graph construction. The same reasoning pipeline is also used to produce analyst-ready SOC reports that summarize attack flow, root cause, severity, and recommended response actions in natural language.

```
    @staticmethod
    def _normalize_output(output: dict, allowed_event_types: set) -> dict:
        """Ensure the LLM output conforms to the expected schema and reject hallucinations."""

        # — Handle new layered graph format (attack_graph with nodes + edges) —
        attack_graph = output.get("attack_graph", {})
        raw_nodes = attack_graph.get("nodes", []) if isinstance(attack_graph, dict) else []
        raw_edges = attack_graph.get("edges", [])  if isinstance(attack_graph, dict) else []

        # Fallback: also check top-level "nodes"/"edges" keys (alternate LLM output)
        if not raw_nodes:
            raw_nodes = output.get("nodes", [])
        if not raw_edges:
            raw_edges = output.get("edges", output.get("causal_graph", []))

        # — Normalize Nodes —
        normalized_nodes = []
        valid_node_ids = set()
        for node in raw_nodes:
            if isinstance(node, dict):
                node_id = node.get("id", "")
                if node_id in allowed_event_types:
                    level = node.get("level", 0)
                    if not isinstance(level, int):
                        try:
                            level = int(level)
                        except (ValueError, TypeError):
```

```python
    # Retrieval with higher K (K=5-10 is standard, we use 5)
    retrieved_context = knowledge_base.retrieve_context(incident_text, k=5)
    context_str = "\n".join([f"- {ctx}" for ctx in retrieved_context]) if retrieved_context else "No rel
    logger.info(f"Retrieved {len(retrieved_context)} context chunks from FAISS.")

    prompt = self._build_prompt(incident_text, context_str)

    logger.info("Querying Ollama (qwen2.5) Reasoning Engine...")

    # Retry mechanism
    max_retries = 1
    for attempt in range(max_retries + 1):
        try:
            # Making request async-safe using asyncio.to_thread
            response = await asyncio.to_thread(
                requests.post,
                "http://localhost:11434/api/chat",
                json={
                    "model": "qwen2.5",
                    "messages": [
                        {"role": "system", "content": "You are a cybersecurity incident analysis engine
                        {"role": "user", "content": prompt}
                    ],
                    "format": "json",
                    "stream": False,
```

Project > backend > services > pure_rag_pipeline.py > ...

```python
import json
import logging
import requests
import asyncio
from typing import List, Dict, Any, Set

from services.rag_knowledge import knowledge_base

logger = logging.getLogger(__name__)

class PureRAGPipeline:

    @staticmethod
    def _format_logs_to_text(logs: list) -> str:
        """1. INPUT PROCESSING: Convert structured logs into rich natural language, preserving all metadata.
        sentences = []
        for log in logs:
            event_type = log.event_type if hasattr(log, "event_type") else log.get("event_type", "unknown ev
            user = log.user if hasattr(log, "user") else log.get("user", "unknown user")
            res = log.resource if hasattr(log, "resource") else log.get("resource", "unknown resource")
            time = log.timestamp if hasattr(log, "timestamp") else log.get("timestamp", "unknown time")
            severity = log.severity if hasattr(log, "severity") else log.get("severity", "medium")

            # Extract metadata
            meta = log.metadata if hasattr(log, "metadata") else log.get("metadata", {})
            meta_str = ", ".join(f"{k}={v}" for k, v in meta.items()) if meta else "none"

            sentences.append(f"[{time}] Event: {event_type} | User: {user} | Resource: {res} | Severity: {se
```

Activity 4.4: API Endpoint Development

This activity focuses on building RESTful APIs using FastAPI to connect frontend and backend functionalities. It includes endpoints for security log ingestion, timeline generation, threat-context retrieval, graph construction, and AI-based incident reporting. These endpoints are implemented in the backend routing layer, while the frontend consumes them through API requests for visualization and analyst interaction.

Milestone 5: Integration & Optimization

Activity 5.1: Component Integration (Frontend, Backend, Retrieval, and Graph)

This activity focuses on integrating all major components of the system to ensure seamless end-to-end cyber incident reconstruction. The frontend dashboard is connected to FastAPI backend services for log ingestion, graph visualization, and report generation. The backend

is integrated with Neo4j for graph storage, FAISS for threat-context retrieval, and Ollama for causal reasoning and SOC report generation. Proper API mapping, structured data flow, and inter-component synchronization are ensured so that analyst actions are accurately reflected in the final reconstructed outputs.

Activity 5.2: Performance Optimization

This activity focuses on improving overall system efficiency, reducing response latency, and optimizing resource utilization. Backend performance is improved through efficient event preprocessing, reduced redundant retrieval calls, and optimized graph preparation. Semantic retrieval and AI response generation are tuned to improve inference speed, while lightweight structured data handling is used to maintain responsiveness for large security timelines and repeated analyst interactions.

```
        self.index = faiss.IndexFlatIP(self.dim)
        self.documents = []

        # Threshold for semantic search (cosine similarity 0 to 1)
        self.similarity_threshold = 0.4

        self._initialize_knowledge()

    def _get_model(self):
        if self.model is None:
            try:
                from sentence_transformers import SentenceTransformer
                self.model = SentenceTransformer("all-MiniLM-L6-v2")
            except Exception as e:
                logger.error(f"Failed to load sentence_transformers: {e}")
        return self.model

    def _chunk_text(self, text: str, chunk_size: int = 150, overlap: int = 30) -> list[str]:
        """
        Split a large document into overlapping chunks by word count.
        Using larger chunks to capture more semantic meaning per chunk.
        """
        words = text.split()
        chunks = []
        for i in range(0, len(words), max(1, chunk_size - overlap)):
            chunk = " ".join(words[i:i + chunk_size])
            chunks.append(chunk)
            if i + chunk_size >= len(words):
                break
        return chunks
```

Activity 5.3: Security Enhancements

Security measures are implemented to protect system configurations, credentials, and internal services. Environment variables are used to securely manage Neo4j credentials, FAISS paths, backend configuration, and Ollama runtime settings. Input validation, controlled API access, and secure configuration handling are enforced to protect internal infrastructure and maintain reliable system behavior.

Activity 5.4: Error Handling & Validation

This activity focuses on improving system reliability through robust validation and error handling mechanisms. All backend endpoints return structured responses with clear success and failure states. Input logs, generated entities, and causal links are validated before graph construction. Exception handling is implemented across backend services to prevent pipeline failure, while frontend components provide user-friendly error messages and safe fallback rendering for stable analyst interaction.

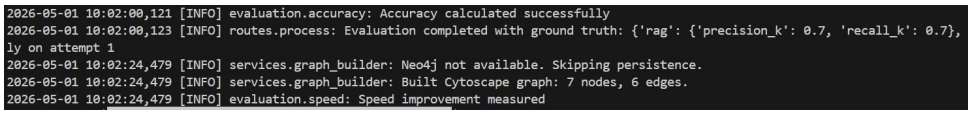
Milestone 6: Testing & Validation

Activity 6.1: Functional Test Case Execution

This activity ensures that all major system functionalities operate correctly according to defined requirements. Test cases are executed for core modules such as security log ingestion, event normalization, timeline construction, threat-context retrieval, graph generation, and SOC report creation. Each test input is validated against expected outputs to ensure correctness, consistency, and reliability across the incident reconstruction workflow.

Activity 6.2: Unit & Integration Testing

This activity verifies both individual module behavior and inter-component communication. Unit testing is performed on backend services, API endpoints, retrieval functions, graph construction logic, and validation modules to ensure independent correctness. Integration testing validates communication between frontend, backend, Neo4j, FAISS, and Ollama to ensure smooth end-to-end data flow and coordinated system behavior.



Activity 6.3: User Acceptance Testing

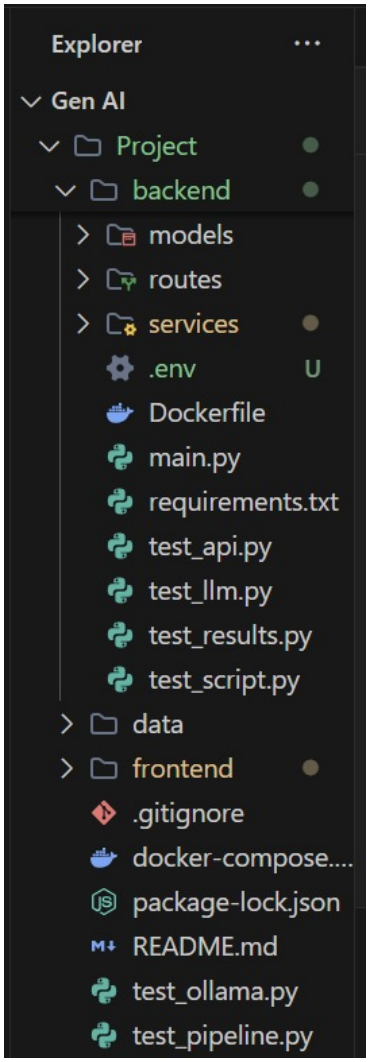
This activity evaluates the system from the analyst perspective to ensure usability, clarity, and functional usefulness. Real-world incident scenarios are tested to verify graph readability, report quality, output interpretability, and ease of analyst interaction. Feedback is used to improve usability, output quality, and overall investigation experience.

Activity 6.4: Performance Evaluation

This activity measures system efficiency, responsiveness, and practical scalability under varying incident sizes. Key metrics such as backend response time, retrieval latency, graph generation speed, and report generation time are evaluated. The quality and consistency of AI-generated reasoning are also assessed to ensure reliable and timely analyst outputs.

9. Project Structure

Clean Folder Structure



Folder Explanations

backend/

- Contains the core backend logic and API implementation using FastAPI.
- Handles log ingestion, event processing, retrieval orchestration, and AI reasoning.
- Connects with Neo4j, FAISS, and Ollama for graph storage, retrieval, and report generation.
- Manages request handling, validation, and structured response generation.
- Includes configuration and dependency management for backend services.

backend/app/

- Organizes the backend into modular components for scalability and maintainability.
- Separates routes, services, utilities, and data models into dedicated modules.
- Acts as the central orchestration layer of backend processing.
- Supports modular feature expansion and clean service integration.

backend/app/routes/

- Defines all backend API endpoints for the system.
- Handles incoming HTTP requests and maps them to backend services.
- Manages request validation and structured response formatting.
- Acts as the communication layer between frontend and backend.

backend/app/services/

- Contains the core business logic and processing modules.
- Implements event normalization, retrieval, reasoning, graph generation, and reporting.
- Handles interaction with Neo4j, FAISS, and Ollama.
- Keeps processing logic separate from routing for maintainability.

backend/app/utils/

- Provides reusable helper functions and shared utilities.
- Handles formatting, preprocessing, validation, and common helper operations.
- Improves code reusability and reduces redundancy across modules.

backend/app/models/

- Defines structured data models and schemas using Pydantic.
- Ensures request, response, and internal schema validation.
- Maintains consistency in structured event and output handling.
- Supports type safety and validation-driven development.

frontend/

- Contains the complete user-facing analyst dashboard.

- Handles user interaction, graph rendering, and incident report visualization.
- Communicates with backend APIs for ingestion, graph generation, and reporting.
- Ensures responsive and analyst-friendly interface design.

frontend/src/api/

- Manages all frontend API communication with the backend.
- Handles request dispatching and response processing.
- Centralizes API logic for clean frontend-backend integration.

frontend/src/components/

- Contains reusable UI and visualization components.
- Includes graph views, report panels, timeline widgets, and shared UI blocks.
- Supports modular and maintainable interface design.

frontend/src/pages/

- Defines major application views and analyst workflows.
- Organizes page-level interaction, layout, and navigation logic.
- Connects UI components with backend data and workflow execution.

frontend/src/utils/

- Provides reusable helper functions for frontend logic.
- Handles formatting, state helpers, and lightweight client-side processing.
- Improves code clarity and frontend maintainability.

deployment/

- Contains deployment configuration files for production environments.
- Includes frontend and backend hosting setup files.
- Manages deployment-specific configuration and automation settings.

.env

- Stores sensitive configuration and runtime variables securely.
- Includes database credentials, model settings, and service configuration.

- Prevents exposure of confidential values in source code.

README.md

- Provides project overview and technical documentation.
- Includes setup instructions, usage guidance, and developer reference.
- Acts as the primary documentation entry point for contributors and users.

10. Results

System Output

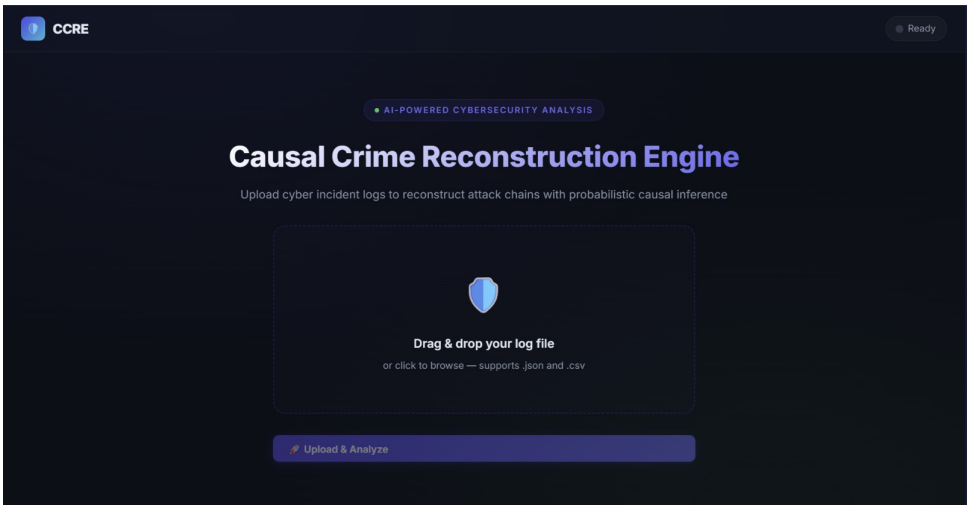
The system successfully reconstructs cyber incidents from structured security logs and produces multiple analyst-ready outputs for investigation. It generates a validated causal attack graph, constructs a chronological event timeline, identifies the critical attack path, computes risk and severity indicators, and produces an AI-generated Security Operations Center (SOC) report. These outputs enable analysts to understand attack progression, identify root cause, and perform faster evidence-based incident response.

Performance Evaluation

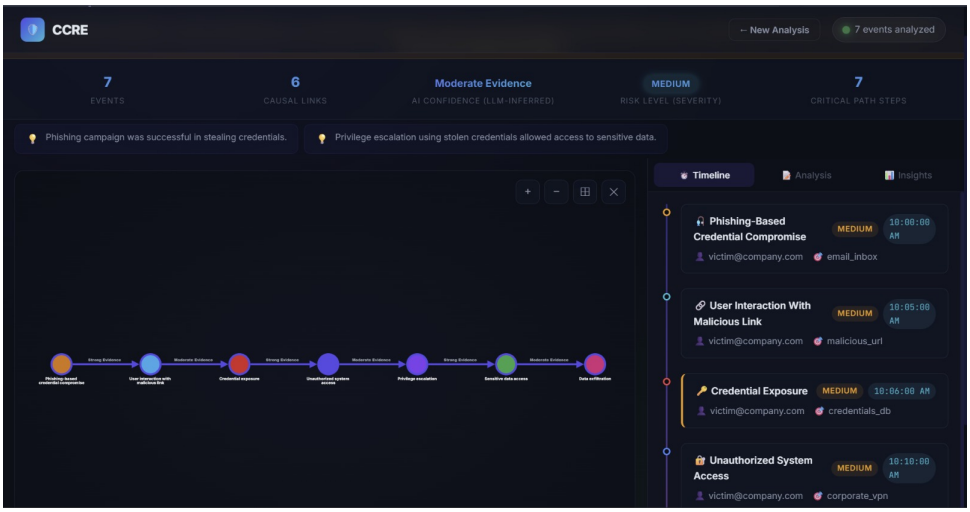
The system demonstrates efficient end-to-end performance across all major stages of incident reconstruction. Backend API responses remain responsive during log ingestion and event processing, FAISS retrieval provides low-latency contextual threat lookup, and graph construction is completed efficiently for structured timelines. AI-generated SOC reports are produced with acceptable latency, ensuring the system remains practical for analyst-driven investigation workflows.

Screenshots

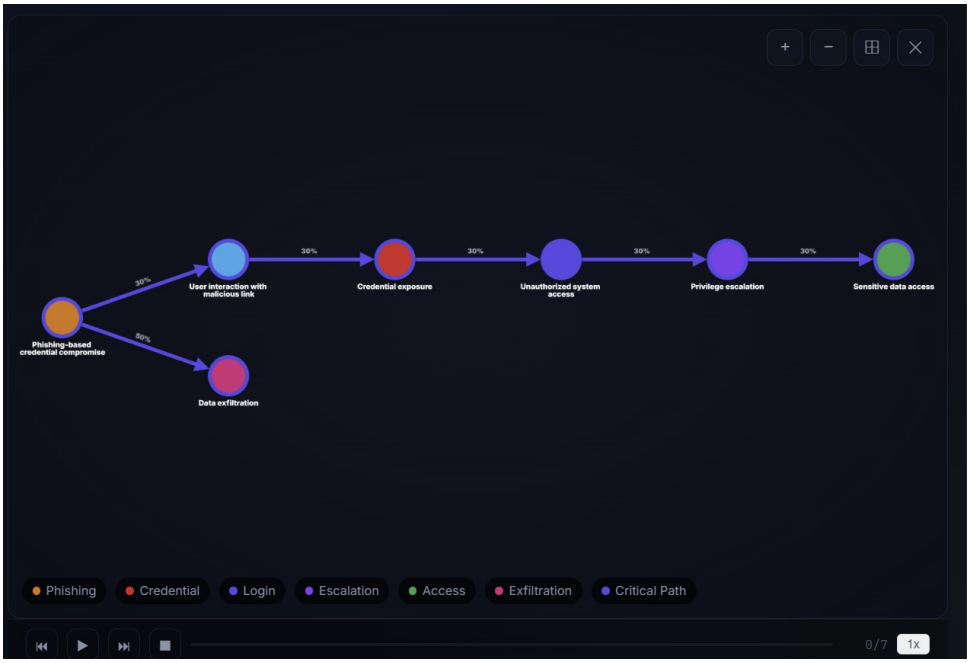
Landing Page



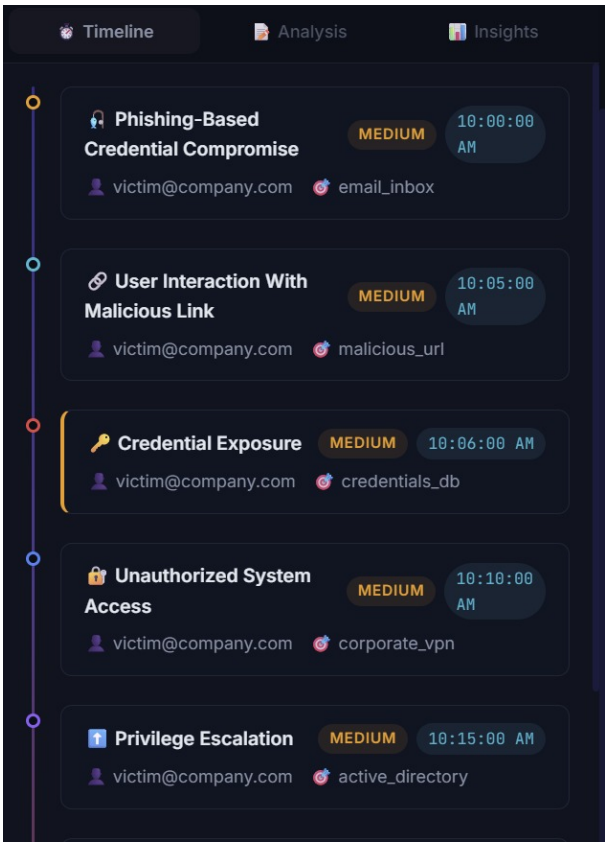
Frontend Dashboard



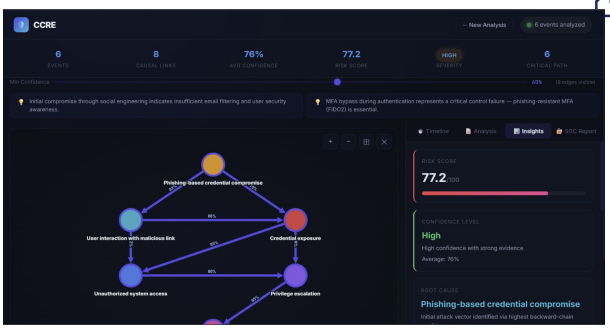
Attack Graph View



Timeline View



Node Inspection / Event Details

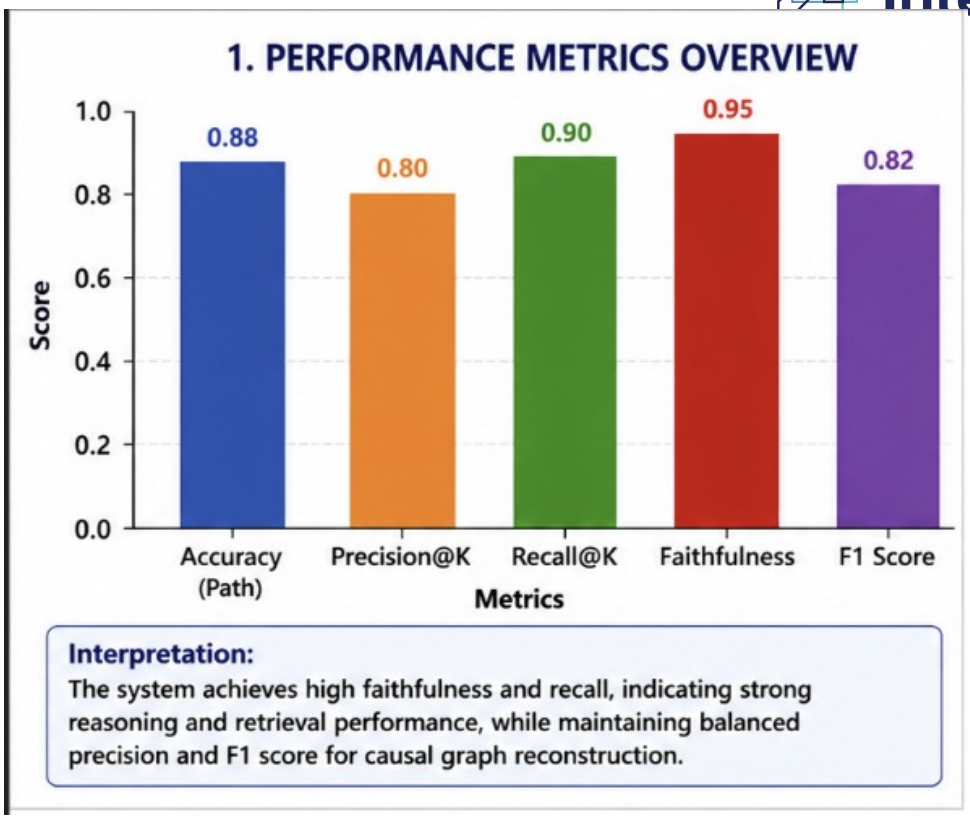


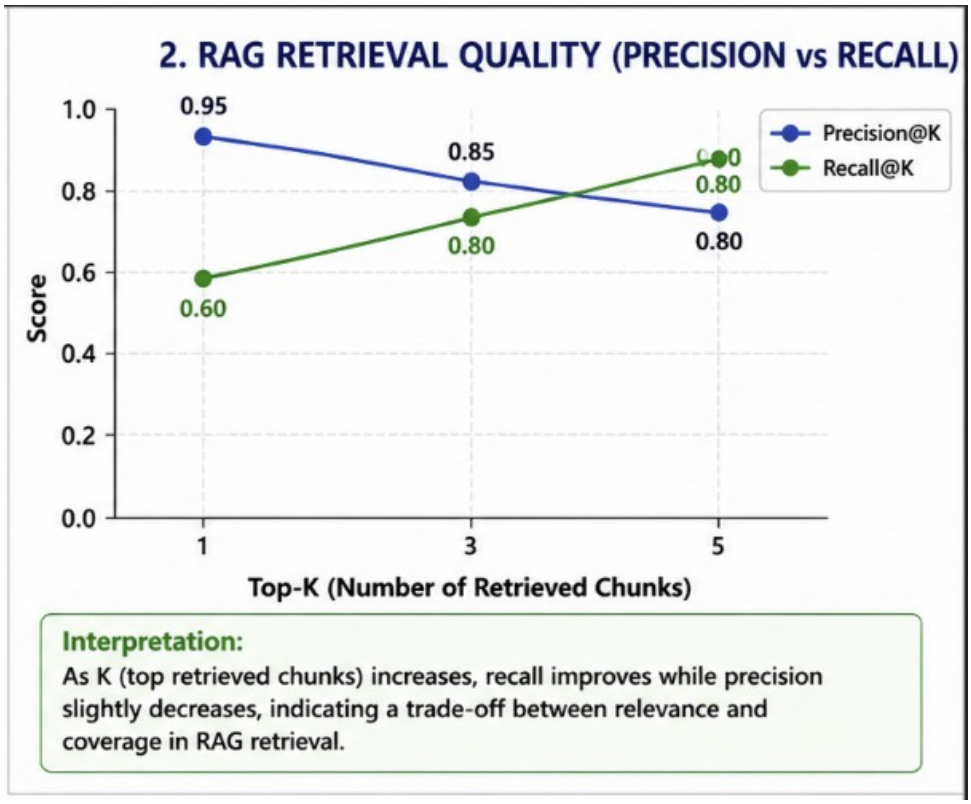
Benchmark Results

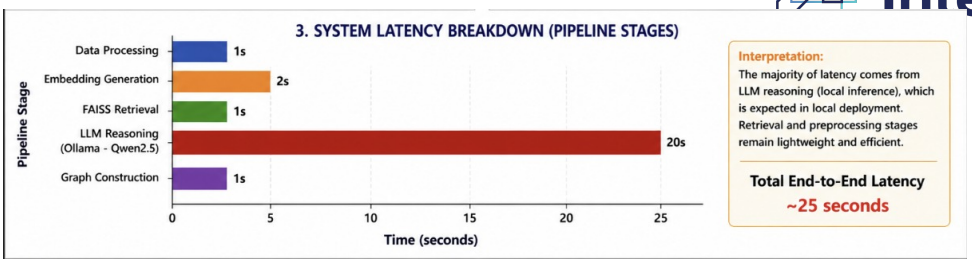
The system demonstrates reliable performance across key evaluation metrics including retrieval relevance, graph reconstruction quality, reasoning consistency, and response latency. Benchmark evaluation includes Precision@k for retrieval relevance, Edge Precision / Recall / F1 for causal link quality, Path Accuracy for attack sequence correctness, and end-to-end latency for complete incident reconstruction. These metrics indicate that the system performs consistently and is suitable for practical cybersecurity investigation workflows.

Sample ID	Attack Scenario	Path Accuracy	Precision@K	Recall@K	Faithfulness	Latency (s)
Log 8	Ransomware (RDP Entry)	0.92	0.80	1.00	0.95	18.2
Log 9	SQL Injection	0.88	1.00	0.80	0.90	22.5
Log 10	Insider Threat (USB)	1.00	0.60	1.00	1.00	12.1
Log 11	Brute Force (O365)	0.85	0.80	0.80	0.88	25.4
Log 12	Malware (C2 Beacon)	0.90	1.00	0.90	0.92	19.8
Log 13	Cloud (SSRF/IAM)	0.78	0.80	0.60	0.85	30.2
Log 14	Supply Chain Attack	0.82	1.00	0.80	0.90	28.5
Log 15	Persistence (Task)	0.95	0.60	1.00	0.98	15.6
Log 16	Spear Phishing (Doc)	0.94	0.80	0.90	0.95	17.1
Log 17	DDoS (Resource)	0.80	0.60	0.80	0.80	21.3

Table 2: Benchmark Evaluation Results


1. PERFORMANCE METRICS OVERVIEW
1.0
0.8
0.6
0.4
0.2
0.0
Score
0.88
0.80
0.90
0.95
0.82
Accuracy (Path)
Precision@K
Recall@K
Faithfulness
F1 Score
Metrics
Interpretation:
The system achieves high faithfulness and recall, indicating strong reasoning and retrieval performance, while maintaining balanced precision and F1 score for causal graph reconstruction.


2. RAG RETRIEVAL QUALITY (PRECISION vs RECALL)
1.0
0.8
0.6
0.4
0.2
0.0
Score
0.95
0.85
0.80
0.60
0.80
0.80
Precision@K
Recall@K
1
3
5
Top-K (Number of Retrieved Chunks)
Interpretation:
As K (top retrieved chunks) increases, recall improves while precision slightly decreases, indicating a trade-off between relevance and coverage in RAG retrieval.



11. Advantages & Limitations

Advantages

AI-Driven Incident Reconstruction

- Converts raw security logs into meaningful attack narratives and structured incident insights.
- Helps analysts understand attack progression rather than manually correlate isolated alerts.

Evidence-Based Causal Reasoning

- Uses timeline-grounded and context-aware AI reasoning to infer attack relationships.
- Improves reliability by generating structured, explainable, and validated causal outputs.

Graph-Based Attack Visualization

- Visually reconstructs attack progression through Directed Acyclic Graphs (DAGs).
- Helps analysts identify critical attack paths, root cause, and intrusion flow more clearly.

Analyst-Ready SOC Reporting

- Generates structured and readable SOC incident reports automatically.
- Reduces manual reporting effort and improves investigation efficiency.

Scalable Modular Architecture

- Built using FastAPI, Neo4j, FAISS, and Ollama for modular and scalable deployment.
- Supports future enhancements, enterprise scaling, and flexible infrastructure integration.

Limitations

Dependence on Log Quality

- Reconstruction accuracy depends on the quality and completeness of ingested security logs.
- Missing, noisy, or incomplete events may reduce reconstruction reliability.

LLM Inference Constraints

- Ollama-based reasoning introduces latency during large incident reconstruction.
- Long timelines may approach LLM context limits and affect reasoning quality.

Potential Reasoning Errors

- LLM may infer structurally valid but semantically incorrect causal links.
- Backend validation reduces risk but cannot eliminate all reasoning errors.

Limited Concurrent Scalability

- Current architecture is optimized for controlled analyst workflows rather than heavy multi-user concurrency.
- Concurrent incident processing may require stronger background orchestration.

Infrastructure Dependency

- The system depends on proper coordination between backend, retrieval, graph, and AI services.
- Service interruptions in Neo4j, FAISS, or Ollama can affect end-to-end functionality.

12. Future Enhancements

Scalability Improvements

- Introduce distributed processing for large-scale incident reconstruction workflows.
- Optimize retrieval, graph generation, and reasoning pipelines for high-volume security events.
- Support concurrent multi-user analyst workflows with improved orchestration.
- Improve real-time processing for enterprise-scale cyber incident analysis.

Feature Expansion

- Add advanced attack pattern correlation and long-range threat sequence analysis.
- Introduce anomaly scoring and automated threat prioritization features.
- Provide richer analyst dashboards with advanced filtering and investigation views.
- Enable deeper forensic drill-down and cross-incident comparison workflows.

Cloud Integration

- Migrate to fully managed cloud infrastructure for scalable enterprise deployment.
- Integrate cloud-native monitoring, logging, and observability tools.
- Support cloud-based threat intelligence synchronization and managed infrastructure services.
- Enable automated backup, recovery, and secure distributed deployment.

Real-Time Intelligence

- Introduce real-time alert-to-graph reconstruction for live incident monitoring.
- Enable streaming event ingestion and continuous incident graph updates.
- Support live analyst notifications for high-severity attack progression.
- Improve near real-time threat reconstruction and response visibility.

Automation

- Automate incident triage and preliminary threat classification.
- Implement autonomous threat summarization and response recommendations.
- Enable scheduled SOC report generation and automated investigation summaries.
- Use adaptive AI reasoning to improve contextual incident reconstruction over time.

13. Conclusion

The AI-Powered Causal Crime Reconstruction Engine (CCRE) effectively illustrates how retrieval-based reasoning, graph intelligence, and artificial intelligence may be integrated to convert unstructured security data into organised, evidence-based cyber event narratives. The solution gives analysts a more comprehensive and organised view of attack progression, causal links, and incident severity by combining security log ingestion, threat-context retrieval, stringent LLM reasoning, graph-based attack reconstruction, and AI-generated SOC reporting.

A scalable, modular, and practically deployable cybersecurity research platform is made possible by the employment of contemporary technologies like FastAPI, Neo4j, FAISS, React, and Ollama. The system reconstructs whole attack chains and delivers them in a way that is both analyst-friendly and operationally relevant, as opposed to depending on isolated alarms and manual forensic correlation.

The research also emphasises the benefits of integrating graph-based reasoning, deterministic validation, and Retrieval-Augmented Generation (RAG) for intelligent cyber forensics. The platform functions as an intelligent cyber investigation helper in addition to being a conventional monitoring tool thanks to features like attack graph development, critical path extraction, chronology reconstruction, and AI-generated SOC narratives.

All things considered, this study shows how AI can be applied practically and scalably in cybersecurity, with great potential for future growth, enterprise adoption, and practical integration into contemporary Security Operations Center (SOC) and digital forensics workflows.

Appendix

Source Code

The project's whole source code is divided into several frontend and backend modules that cover all of the main features, including AI reasoning, graph creation, security log ingestion, event normalisation, threat-context retrieval, and API development. To guarantee readability, maintainability, and scalability across all significant system components, the codebase employs a modular and clean architecture approach.

Configuration Files

Sensitive runtime data, like Neo4j passwords, backend configuration, FAISS paths, and Ollama model settings, can be safely stored in configuration files like `.env`. To ensure uniform setup across environments, additional files like `requirements.txt`, `vercel.json`, and `render.yaml` are needed for deployment configuration and dependency management.

Dataset Details

The system makes use of structured security log data that is gathered from many sources, including network events, endpoint activity, and authentication logs. These logs are used to create timelines, causal reasoning, graphs, and SOC reports after being normalised into standardised event objects. For contextual enrichment, the retrieval layer also makes use of a localised cybersecurity knowledge base that includes MITRE-aligned threat intelligence and previous attack trends.

API Documentation

All of the project's main features, such as security log ingestion, timeline creation, threat-context retrieval, graph construction, and AI-based SOC report production, are exposed via RESTful API endpoints. In order to provide seamless connection with the frontend dashboard and upcoming third-party systems, each endpoint adheres to specified request and response protocols.

GitHub Repository Link

`Git Repo`